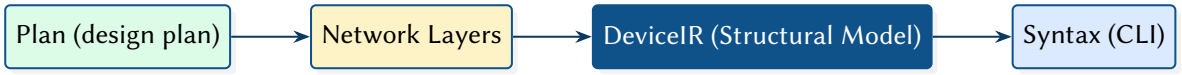
NetCfg: A Declarative Network Configuration Compiler

Formal Methods in Configuration Synthesis

Simon Knight, Ph.D.

March 2026 • Version 4.0.0

Last updated: March 13, 2026



Abstract. NetCfg is an architectural network configuration engine that takes a Blueprint (annotated topology)—the "source code" of the network—and compiles it into verified, target-specific device configurations. A companion architectural plan selects which compilation strategies to apply. The synthesis engine processes these inputs through a five-layer pipeline: (1) Intent Transformation, (2) Topological Validation, (3) Lowering to a vendor-neutral Device IR, (4) Target-Specific Device Model Transformation, and (5) Syntax Serialisation. This report documents the data model, the transformation DSLs, and the structural model-based lowering mechanism that decouples architectural logic from physical hardware constraints.

Contents

1 Introduction

Large-scale network configuration is a compilation problem.

Insight: *Manual CLI entry is the "assembly language" of networking; NetCfg provide the high-level language and compiler.*

Shenker [1] diagnosed the problem: networking adds protocols rather than abstractions. This produces a "bag of protocols" with no composable interfaces. He proposed three abstractions to decouple intent from mechanism: a forwarding interface, a global network view, and a specification language.

Gill et al. [2] quantified the cost: misconfiguration causes most customer-impacting incidents in Microsoft data centres, usually triggered by operator error. As data centres grow to millions of lines of config, the need for automated, intent-driven synthesis increases. Yet most automation remains bespoke and non-transferable.

Recently, large-scale topology systems [3] have introduced *multi-abstraction-layer topology* modelling. A shared representation spanning physical, logical, and design layers unifies network operations. The key insight—that "almost all other network-management data either derives from its topology, constrains how to use a topology, or associates resources with specific places"—motivates NetCfg's graph-centric design.

Meanwhile, model-driven management via OpenConfig [4] and YANG [5] has standardised vendor-neutral *per-device* data schemas, but these models do not capture cross-device topology relationships or support graph-wide operations like IP allocation.

: Hardware Independence

Architectural logic must lower to a vendor-neutral Intermediate Representation (DeviceIR) before any target-specific syntax is generated. This decouples policy intent from physical ASIC constraints.

What NetCfg is

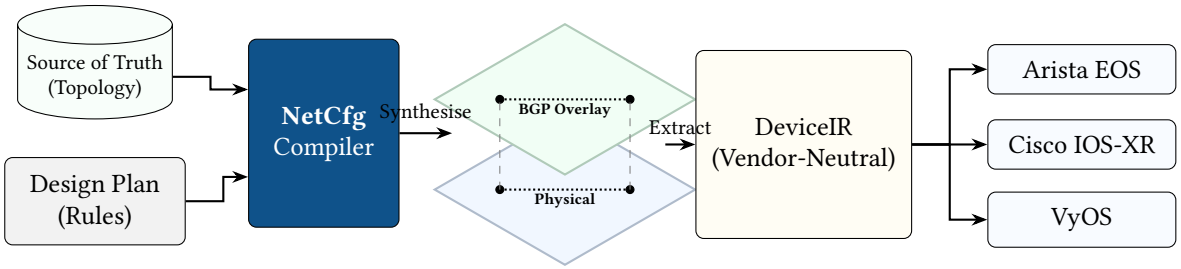


Figure 1. NetCfg acts as the compilation bridge between Source of Truth systems and physical network hardware. Powered by its declarative query language, it synthesises the physical inventory and design rules into logical **Network Layers** (e.g., BGP, OSPF). This allows operators to validate their architectural intent *before* extracting the Vendor-Neutral Intermediate Representation (DeviceIR).

NetCfg is a command-line tool and synthesis engine that compiles an *Blueprint (annotated topology)* into per-device configuration files. The annotated topology is the primary input: a graph of devices and links enriched with semantic properties—roles, sites, address pools, protocol flags, trust levels—that encode the operator's intent at a network-wide level. A companion *design plan* selects which compilation strategies (meshing patterns, allocation policies, assertion rules) to apply when deriving device-level state from those annotations. Optionally, a *target backend* (mapping rules and vendor templates) describes how to extract vendor-specific configuration stanzas from the enriched topology. The synthesis engine produces deterministic, diffable, auditable configuration artefacts for every device in the network.

NetCfg replaces the manual "glue" layer between source-of-truth systems (NetBox [6], Nautobot [7]), automation frameworks (Ansible [8], Nornir [9]), and network modelling tools (Batfish [10]). It treats network configuration as a *compilation problem*: topology intent is compiled through a typed pipeline, not interpolated through ad-hoc templates.

> **What NetCfg is *not***
>
> To understand the architecture, define its boundaries:
> - **It is not a Source of Truth.** It does not store inventory or manage IPAM databases. It *consumes* data from systems like NetBox and projects it into a layered graph.
> - **It is not a Deployment Engine.** It does not SSH into routers or push configuration via NETCONF/gNMI. It *compiles* the declarative artifacts that tools like Ansible then deploy.
> - **It is not a Data-Plane Verifier.** It does not simulate packet flow like Batfish. It proves *structural intent* (e.g., "Are these zones isolated in the model?") rather than simulating routing tables.

Where OpenConfig standardises the *schema* of device state, NetCfg standardises the *derivation* of that state from topology intent. Earlier systems provided a shared topology *representation*; NetCfg provides a topology *compiler* that transforms intent into per-device configuration.

Design philosophy

Three principles explain every design decision in this document:

1. **Lowering over templating.** The operator's intent is lowered through successive abstraction stages, mirroring the multi-stage lowering architecture of LLVM [11]. LLVM lowers high-level source through a language-independent IR (LLVM IR) to target-specific machine code; NetCfg lowers topology intent through a vendor-neutral DeviceIR to platform-specific CLI syntax. The analogy is precise: LLVM's frontend/IR/backend separation corresponds to NetCfg's design plan/DeviceIR/template separation. Hardware quirks and vendor-specific logic are handled via structured Model-to-Model transformations, not ad-hoc string concatenation.
2. **Separation of what from how.** The Blueprint (annotated topology) declares *what* the network contains—devices, links, and their semantic properties. The design plan declares *which* design rules to apply when compiling that topology (meshing style, allocation strategy, assertion constraints). The mapping and templates declare *how* to render the enriched model into per-device configuration. Platform-specific rules are injected via `imports: ['hardware-lowering.yaml']` and isolated behind `when: device_os == ...` predicates. The same topology and design plan can produce Cisco IOS, Arista EOS, or Juniper JunOS output by swapping only the mapping and templates.
3. **Topology as the single source of truth.** Topology is the root from which all other network-management data derives. NetCfg treats the graph as the canonical representation. IP addresses, protocol sessions, policy rules, and security zones are *computed* from topology relationships, not declared independently.

Audience

This document is intended for:

- Engineers integrating NetCfg into a network automation pipeline.
- Contributors to the NetCfg and NTE codebases.
- Researchers evaluating graph-based approaches to network configuration.

Familiarity with YAML, basic graph theory, and IP networking is assumed. Experience with programming is helpful but not required to extend the engine.

> **Companion report: NTE-TR-001**
>
> NetCfg implements the compilation leg of the deployment strategy defined in NTE-TR-001 §Core Architectural Insights. NTE provides the graph substrate; NetCfg compiles annotated

topologies into verified device configurations. NetCfg operates exclusively in NTE's Strict Mode (design-authority, schema-validated compilation).

How to read this document

This report is organised into three parts. **Part I** (Architecture & Insights) introduces the conceptual model, the network data model, intent and policy expression, and the compilation pipeline—purely conceptual, with no implementation detail. **Part II** (The Design Plan Language) specifies the declarative surface: plan structure, selector syntax (NTE-QL), named groups and objects, the primitive catalogue and target-specific transforms. **Part III** (Engine Implementation) covers the pipeline internals, core mechanisms (IPAM, protocol overlay, DiffEngine, TSDM), the CLI, an end-to-end worked example, the error model, editor integration, and current limitations.

1.1 Case Study: From PhD to Production (Architectural Synthesis)

The NTE design philosophy is rooted in the **Architectural Synthesis** strategy developed during PhD research into automated network design. This framework treats network design as a three-stage transformation pipeline where the `ank_netcfg` compiler acts as the synthesizer, lowering high-level intent into the rigorous NTE topology model: **Design Intent → Synthesis Plan → Layered Topology**.

Figure 2. The Architectural Synthesis Pipeline. `ank_netcfg` aggregates intent from abstract human domains into a singular formal plan, which it then algorithmically lowers into the NTE graph representation.

1.1.1 Case Study: Segment Routing (SR-TE) Migration

Consider a large-scale migration from RSVP-TE to Segment Routing (SR-TE) in a service provider core. In the PhD thesis context, this was modeled as a transition between two distinct forwarding paradigms sharing a single physical underlay.

1. The Design Intent. The architect defines high-level constraints: "Gold traffic must avoid PoP-X and use low-latency links." This intent is captured in an abstract specification mapping service classes to topological constraints.

2. The Synthesis Plan. The `ank_netcfg` synthesis engine aggregates these constraints into a singular `SR_Policy_Plan`. In NTE, this is represented as a set of high-level `Semantic` edge declarations targeted for expansion.

```yaml
nodes: [R1, R2, R3, R4]
sr_policies:
  - id: "GOLD-PATH"
    head: R1
    tail: R4
    constraints: { avoid: [PopX], metric: latency }
```

3. The Layered Transformation. The `ank_netcfg` compiler Lowers this Synthesis Plan into the formal NTE graph model:

- **Underlay (L1/L2)**: The physical fiber and IGP adjacencies (IS-IS) are verified in NTE.
- **Shadow Layer**: The synthesis engine instructs NTE to create **Shadow Nodes** for the required SIDs (Segment IDs) if they are not yet discovered.
- **Overlay (SR-TE)**: The `Semantic` edge is lowered into a series of `Parent` edges linking the policy to the underlying physical path.

By using this PhD-derived synthesis framework, `ank_netcfg` allows architects to reason at the "Intent" level while the NTE engine handles the "Production" rigor of cross-layer mapping and verification.

2 The Synthesis Pipeline (Conceptual Model)

This section explains the three ideas underpinning every NetCfg design plan: (1) the network model is a *layered graph*; (2) design plans *compose* the model declaratively, layer by layer; (3) every primitive follows the same *select* → *enrich* → *validate* pattern. Understanding these concepts makes the formal data model (§3) and DSL syntax (§??) easier to absorb.

Keep the mental model, postpone the mechanisms. In this section, treat NetCfg as a compiler operating over a graph: selectors identify where in the graph to act, primitives add derived state, and assertions act as quality gates. The implementation details (engine internals, NTE-QL query compilation, DataFrames) exist to make this model fast and safe, but they are not required to understand the flow.

2.1 The Core Pipeline Taxonomy

NetCfg treats network configuration as a pure compilation problem, driven by a functional synthesis pipeline. Instead of writing per-device stanzas, operators declare design rules that the synthesis engine processes through a sequence of logical stages. This funnel progressively refines high-level abstract intent down to concrete, hardware-specific syntax.

The synthesis engine separates domain knowledge (the network architecture and vendor mappings) from engine mechanics (graph mutations and Structural Model generation). The pipeline enforces this separation via four strict phases:

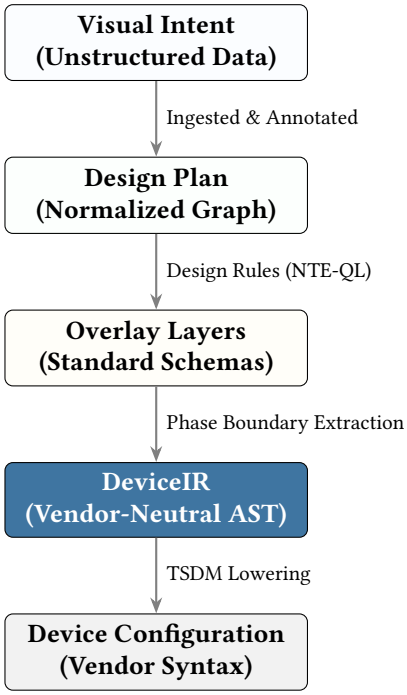


Figure 3. The Core Pipeline Taxonomy. Visual Intent is ingested into the normalized Design Plan graph. NTE-QL Design Rules project nodes into standard Overlay Layers. The result is extracted to DeviceIR and finally lowered into vendor syntax.

This conceptual workflow guarantees that the target configurations perfectly reflect the architectural plan. Because state can only flow forward through the pipeline, the final syntax is strictly a mathematical derivative of the design intent, eliminating the drift and “spaghetti” configuration inherent to manual device-by-device workflows.

Case study: multi-tenant isolation

The multi-tenant data-centre case study (`examples/case_studies/04_multi_tenant_dc.yaml`) illustrates how these concerns compose. The operator declares: (1) groups that name device roles and per-tenant link populations; (2) a hub-and-spoke mesh for the fabric; (3) per-VRF address pools; (4) access policies that deny cross-VRF traffic; (5) assertions that every device has a tenant assignment and that each tenant's subgraph is connected; and (6) a blast-radius limit. At no point does the operator name a specific device or IP address—the compiler derives all concrete state from the declared populations and intent.

Assertions, rules, and primitives can be factored into reusable libraries and shared across design plans via `imports:`. §?? catalogues the standard libraries that ship with NetCfg.

2.1.1 The Compilation Pipeline

This section describes the pipeline conceptually; Part III (§??) covers each layer's implementation.

NetCfg's compilation pipeline has five distinct layers of abstraction, operating as a true network compiler. Figure 4 shows the data flow from abstract intent to concrete syntax.

A key property of this pipeline is **determinism**: configuration is a pure function $f(\text{Annotated Topology}, \text{design plan}) = \text{DeviceIR} \rightarrow \text{Config}$. If neither the topology annotations nor the design plan strategies have changed, regenerating the configuration produces a bit-identical result, making any divergence immediately detectable via diff. Combined with the structural diff engine (§??), this enables a "regenerate and compare" workflow for detecting configuration drift without runtime monitoring.

Note

The conference paper describes six implementation stages (parse, shadow, execute, map, render, write); this report groups these into five conceptual layers (Intent, Assertions, Mapping/DeviceIR, TSDM, Syntax). Both describe the same pipeline at different levels of abstraction: the conference paper's "parse" and "shadow" stages are subsumed by Layer 1 (Intent Transformation), and "render" and "write" are subsumed by Layer 5 (Syntax Serialisation).

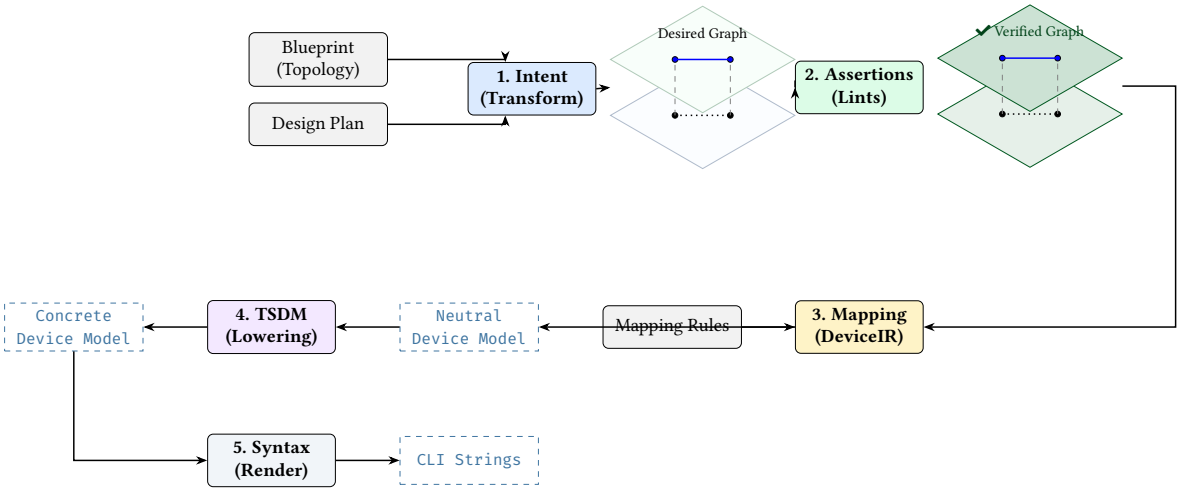


Figure 4. NetCfg compilation pipeline. High-level intent is progressively lowered through three distinct mathematical graph representations (Verified Graph, Neutral Device Model, Concrete Device Model) before serialisation.

3 The Topological Ontology (Data Model)

To synthesize configuration across a multi-vendor landscape, NetCfg relies on a highly structured internal data model. This section outlines the principles of this topological ontology.

3.1 The Fallacy of the Simple Graph

In classic computer science graph theory, a network is often modeled as a simple graph $G = (V, E)$, where V are routers and E are the cables connecting them. This is a massive trap that has caused countless network automation tools to fail at scale.

In reality, cables do not plug into "routers"; they plug into interfaces, which are hosted on transceivers, which reside on line cards, which are inserted into chassis slots. To accurately model physical realities and ensure deterministic algorithmic traversal, NetCfg abandons the naive $G = (V, E)$ approach. Instead, it enforces a strict **Endpoint Ontology**: Nodes connect via Structural edges to Endpoints, and Endpoints connect via Connectivity edges to other Endpoints (`Node` → `Structural` → `Endpoint` → `Connectivity` → `Endpoint` → `Structural` → `Node`). This provides a mathematically uniform canvas for algorithms, freeing them from parsing custom vendor chassis logic.

3.2 The Fractal Nature of Nodes

Network devices are not atomic black boxes; they are networks unto themselves. A single modular core router contains its own internal fabric, line cards, and VRFs. The NetCfg ontology treats topology as fractal: the exact same graph primitives (Nodes and Structural edges) used to build a macro data center topology are used to build the internal containment hierarchy of a single switch. An algorithmic query written to trace a path across a WAN can be used, unmodified, to trace a path through the internal hardware of a router.

3.3 Property Flattening vs. Schema Hell

Most modern network automation tools (like YANG models or NetBox) bury network state in deeply nested, highly proprietary JSON/XML trees. When data is nested deeply inside a device's JSON payload, it is invisible to generic graph algorithms (e.g., a generic "shortest path" algorithm doesn't know how to parse an Arista-specific nested block to find link delay).

NetCfg enforces **Property Flattening**. All domain semantics and state are stored as flat key-value pairs directly on the topological primitives (Nodes and Edges). This decoupling creates an Algorithmic Canvas: engineers can write generic Cypher-based graph queries (e.g., `MATCH (n) WHERE n.status == 'down'`) without needing to understand the underlying schema.

3.4 Orthogonal Layering

Networks do not exist in a single dimension. A physical fibre cut (Layer 1) causes an IP adjacency drop (Layer 3), which drops a BGP session (Layer 4), which withdraws a VPN route (Layer 7).

In NetCfg, these layers are not just tags; they are Orthogonal Graph Planes bound by strict hierarchical dependencies (Parent edges). The entire digital twin is modeled as a **Directed Acyclic Graph (DAG) of Graphs**. This allows the engine to treat the BGP topology as a completely pure, mathematically sound graph for BGP algorithms, while still retaining the deterministic physical lineage back to the hardware.

3.5 Per-node state: DeviceContext

§3.6 explained that all renderable state lives on nodes. Formally, each node's properties are stored as structured metadata in the columnar property store D, keyed by node type. NTE-QL expressions (which compile to Cypher-based graph traversals internally) unpack this payload on demand. The synthesis engine exposes a typed *device context* whose fields are progressively enriched by primitives:

Table 1. Per-node device context: fields populated by the synthesis engine pipeline. Fields marked with † are written by specific primitives; all others originate from the Blueprint (annotated topology).

Property	Architectural Source
`hostname`	Extracted from the digital twin node identity
`device_os`	Declared as an immutable topology annotation
`management_ip`	Declared as an immutable topology annotation
`src_ip`, `dst_ip`, `subnet`	Synthesized dynamically by the `provision_ips` design rule †
`src_ipv6`, `dst_ipv6`, `subnet_v6`	Synthesized dynamically by the `provision_ips` design rule † (dual-stack)
`vrf`	Inherited from topology or dynamically allocated †
`peer_interfaces`	Topologically computed by the `mesh_nodes` design rule †
`stanzas`	The accumulated logical configuration model †
`(any other key)`	Free-form topological metadata or enriched facts

Primitives progressively enrich this context: `mesh_nodes` establishes edges and writes `mesh_type`/`peer_count` into metadata; `provision_ips` writes IP fields; `build_protocol_layer` deep-merges config overlays into the cloned node's metadata. By the time rendering occurs, each node's device context contains all information needed to produce its configuration file.

> **Note**
>
> Any topology annotation that does not match a named field is captured in the node's metadata map. This makes the device context extensible without schema changes—operators can attach arbitrary properties in the topology and reference them from selectors or templates.

> **Note**
>
> **Graph engine dependency.** The NTE topology engine [12] (NTE-TR-001) provides the directed graph with columnar (Polars DataFrame) property storage. NetCfg uses NTE for the graph, the mapping evaluator, and DeviceIR types. NetCfg's `mesh_nodes` and `build_protocol_layer` primitives rely on the Endpoints Model—a 3-hop traversal pattern (NTE-TR-001 §Data Model) that discovers adjacent devices via the DWC (Device–Wire–Cable) abstraction.

3.6 The network model as a Layered Topology

NetCfg represents the network-wide state using a **Layered Topology**. The topology is a graph of *nodes* (devices) and *edges* (links). The overall network state is captured in a stack of these planes, starting from the physical cabling up through the logical protocol overlays.

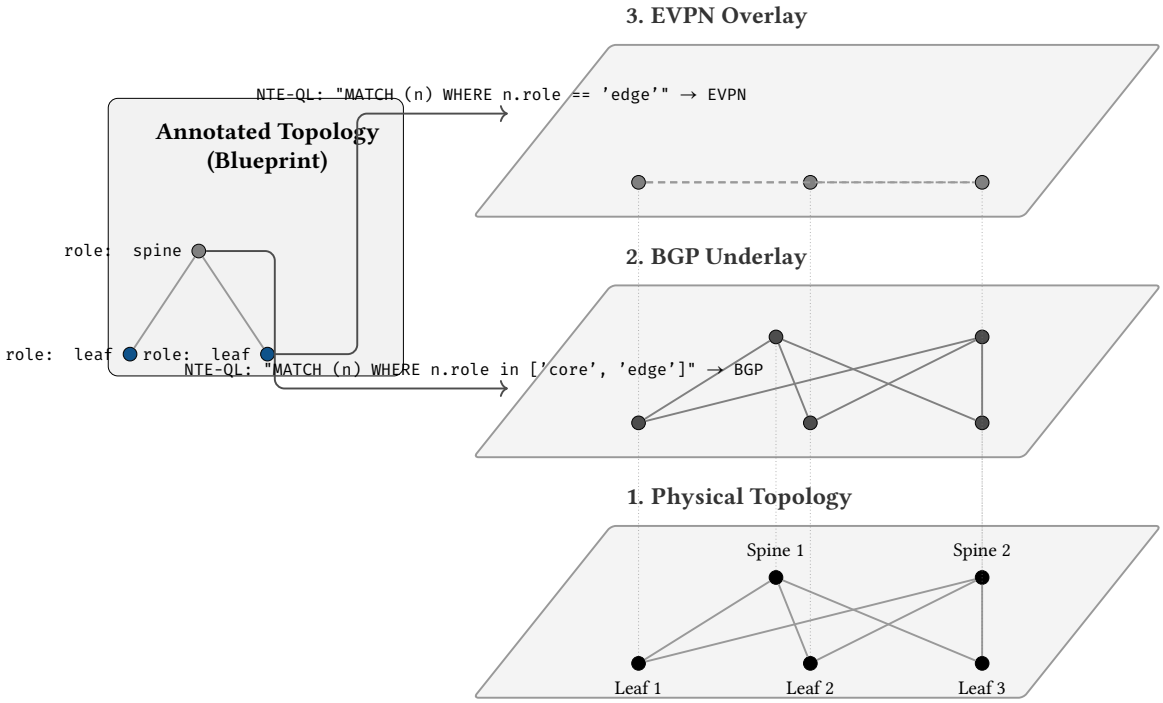


Figure 5. Intent Mapping to Layered Topology. Rather than compiling unstructured templates, operators declare intent in a Design Plan. Primitives in the plan directly map logical intent onto orthogonal topology layers.

Mapping Intent into Layer Projections. A source of confusion in network automation is the distinction between what an operator *declares* and what the engine *computes*. In NetCfg, we use specific terminology:

- **Mapping (Human domain):** The operator writes a Design Plan (YAML). This plan *maps* intent via Primitives (like `build_protocol_layer`) to a target population (like "all leaves").
- **Projection (Engine domain):** The synthesis engine reads the plan and mathematically *projects* nodes from the physical layer upwards into logical protocol layers (e.g. creating BGP nodes from physical nodes).

When a primitive executes, it maps the user's intent directly onto the topology. As nodes are projected into a logical layer, the primitive applies structural *transformations*, merging the intended configuration (such as AS numbers or VPN address families) onto the newly created overlay nodes.

Because every projected node retains a parent mapping back to its physical origin, the logical overlay implicitly inherits the physical truths of the network without redundant data entry. The synthesis engine does not need to separately map IP addresses into EVPN configurations; the EVPN node simply looks down its parent mapping to retrieve the hardware's allocated addressing.

> **Note**
>
> **Design note: logical edges do not mirror physical edges.** Because protocols exist in their own structural planes, their topologies can differ entirely from the physical hardware. An EVPN layer might project a full mesh of VXLAN tunnels between Leaf nodes, ignoring the physical Spines entirely, while an OSPF layer might strictly follow every physical point-to-point cable. Layer projection decouples the logical network architecture from the physical cable plant.

Kinds of layers (a practical taxonomy)

Layers are intentionally lightweight: they let the model hold multiple simultaneous views of the same network without conflating their semantics. In practice, most NetCfg design plans use a small, repeatable taxonomy:

1. **Physical layer.** Nodes represent devices; edges represent physical links. Typical state: interface naming, link addressing, management reachability, hardware inventory.
2. **Protocol layers (routing/switching overlays).** Nodes represent a device's participation in a protocol (one node per device per protocol); edges represent logical adjacencies or sessions. Typical state: neighbours, timers, areas/ASNs, policy attachments, protocol feature flags.
3. **Policy and service layers (optional).** Some organisations model policy and services as dedicated layers when the relationships are graph-shaped (e.g., security-zone bindings, NAT relationships, telemetry subscriptions). Others keep these as node metadata attached to the physical or protocol nodes. The design rule is the same either way: *renderable state stays on nodes; edges express relationships.*

This taxonomy is not prescriptive: the point is that layering preserves separation of concerns while keeping every derived fact traceable back to the physical topology via the parent mapping.

Design rules that build overlays

Layering alone provides representation; *design rules* provide synthesis. In NetCfg, design rules are implemented as primitives that read from the current model (often the physical layer) and then materialise overlay structure and per-device stanzas. They are the bridge from plan topology to protocol overlays.

Two common classes of design rule are:

1. **Overlay construction rules.** `build_protocol_layer` clones selected physical nodes into a protocol layer, records the parent mapping (`_source_id`), and deep-merges protocol defaults into the clones. With `clone_underlying: true`, it also seeds the overlay with a derived copy of the physical connectivity.
2. **Session synthesis rules.** Protocol session primitives (e.g. `build_bgp_session`, `build_ospf_session`, `build_isis_session`)[1] consume the overlay graph and emit configuration stanzas on the participating nodes. These rules are where operator intent becomes concrete neighbours, interfaces, and per-peer parameters.

3.7 Modeling Real-World Protocols

Insight: *The operator does not "mesh in the BGP layer" manually. They declare intent; the design rules materialise the overlay relationships and the resulting per-device stanzas in the appropriate layer.*

While the NTE core is protocol-agnostic, it provides the formal graph primitives and validation invariants required to represent complex, multi-layered networking protocols. External synthesis engines like `ank_netcfg` lower abstract design intent into these specific NTE graph structures. This section catalogs how common protocols are mapped into the NTE engine.

3.7.1 Layer 3 Foundations: OSPF and IS-IS

In the IP routing layer, nodes represent routers or virtual routing instances (VRFs).

Adjacencies are modeled as `Connectivity` edges between `LogicalEndpoint` vertices representing routed interfaces (e.g., /30 or /31 point-to-point links). This supports both OSPF [13] and IS-IS [14] adjacency topologies.

Areas/Levels are modeled as `LogicalNode` vertices. A router participating in OSPF Area 0 has a `Parent` edge to the `Area0` LogicalNode.

LSDB Visibility Protocol-specific state (e.g., OSPF Router IDs, IS-IS System IDs) is stored in the property map of the `Node` or `LogicalEndpoint`.

3.7.2 BGP Control Plane

BGP [15] is modeled as a logical overlay on top of IP reachability.

BGP Sessions are modeled as `Connectivity` edges at the `bgp` layer. For multi-hop eBGP, these edges connect `LogicalEndpoint` vertices (Loopbacks), while for single-hop eBGP, they may connect physical `Endpoint` vertices.

[1] Session synthesis primitives are planned but not yet in the catalogue; the current workflow uses `build_protocol_layer` with a config overlay to achieve the same result.

AS-Path and Communities are stored as columnar attributes on the BGP `Connectivity` edges, enabling high-performance SIMD filtering (e.g., "Find all sessions with AS 65000 in the path").

Route Reflectors are modeled using `Semantic` edges to represent the client-to-RR relationship without requiring explicit endpoint pinning.

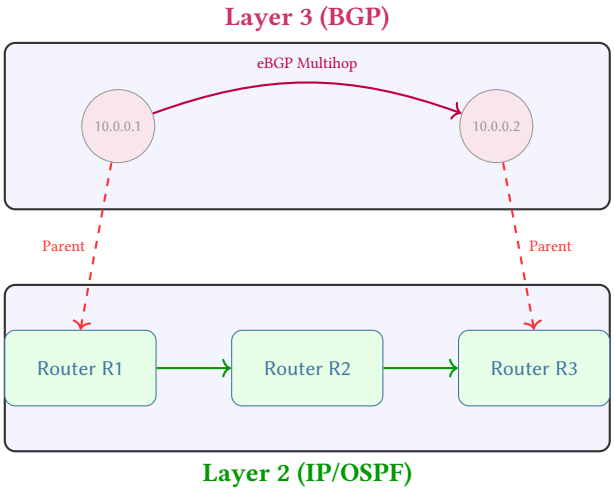


Figure 6. Multi-hop BGP dependency mapping. A logical BGP session at L3 is anchored to IP endpoints on R1 and R3. The session's existence depends on the multi-hop path (R1-R2-R3) in the underlay IP layer.

3.7.3 MPLS and Segment Routing (SR)

Multiprotocol Label Switching (MPLS [16]) introduces label-switched paths (LSPs) that may not follow the shortest IGP path.

Label Switched Paths (LSPs) are modeled as `Semantic` edges at the `mpls` layer. An RSVP-TE tunnel from R1 to R5 is a single Semantic edge with an attribute containing the full `hop_list`.

Segment Routing (SR-TE) is modeled using **Shadow Nodes** for the segment identifiers (SIDs). The engine can verify if an SR policy's SID-list is topologically valid by traversing the underlying physical layer.

LDP/RSVP Sessions are modeled as `Connectivity` edges at the `mpls_control` layer, sharing `Parent` edges with the physical interfaces they run on.

3.7.4 Network Virtualization: VXLAN and EVPN

Modern data center fabrics use VXLAN [17] encapsulation orchestrated by BGP-EVPN [18].

VTEPs (VXLAN Tunnel Endpoints) are modeled as `LogicalEndpoint` vertices on the `vxlan` layer.

VNIs (VXLAN Network Identifiers) are modeled as `LogicalNode` vertices. All VTEPs participating in a specific VNI have a `Parent` edge to that VNI node.

EVPN Control Plane is modeled as a sub-family of the `bgp` layer. MAC/IP reachability is stored as property metadata on the `vxlan` Connectivity edges.

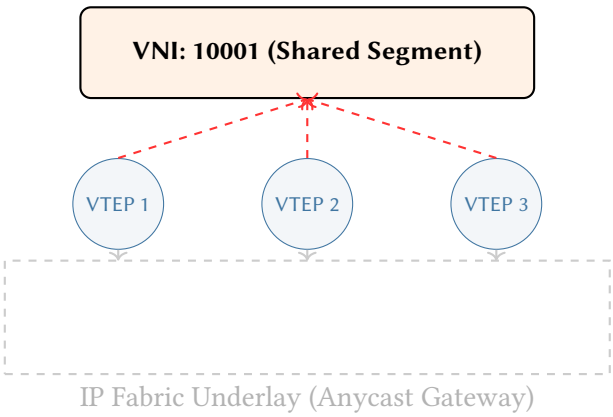


Figure 7. VXLAN VNI as a logical shared segment. Individual VTEPs are mapped to a central `LogicalNode` representing the VNI. This allows NTE-QL to treat the complex underlay fabric as a single broadcast domain for overlay verification.

3.8 Topology provenance

The Blueprint (annotated topology) can enter NetCfg through three channels, each preserving the node annotations that selectors and primitives operate on:

1. **Hand-authored structured formats.** The operator writes the topology file directly, placing each node's annotations under a `properties:` block. This is the most explicit channel and is common during greenfield design or lab work.
2. **Import from existing configurations** (`netcfg import`, §??). Parses a directory of device configurations (or a device inventory file) and produces a topology archive (`.nte`), inferring hostnames, links, and device platforms from the configuration data. Operators then enrich the imported topology with additional annotations (roles, sites, trust levels) before compiling.
3. **NSoT synchronisation** (`netcfg sync pull`, §??). Pulls a live topology from a network source of truth such as NetBox [6] via its GraphQL API, mapping inventory fields to topology annotations. This channel keeps the Blueprint (annotated topology) in sync with the production network.

Regardless of channel, the result is the same data structure: a directed graph with a structured property document per node. Primitives and selectors are agnostic to provenance—a topology imported from NetBox is compiled identically to one hand-authored in YAML.

4 The Design Plan (DSL)

4.1 Three input documents

An operator supplies three documents to the synthesis engine:

1. **Annotated topology** — the "source code" of the network. A graph of devices (nodes) and links (edges), where each node carries rich semantic annotations: `hostname`, `role`, `site`, `device_os`, trust levels, address pools, protocol flags, and any other properties the operator considers part of the network's design intent. The topology may be hand-authored (structured formats), imported from a network inventory system via `netcfg import` (§??), or pulled live from an NSoT such as NetBox [6] via `netcfg sync pull` (§??).
   These annotations are the data that the synthesis engine's selectors (e.g. `MATCH (n) WHERE n.role =='core'`) and primitives operate on. The richer the annotations, the more the synthesis engine can derive automatically. Ideally, the topology captures *all* network-wide design intent; the design plan then reduces to a thin set of compilation strategy choices.
2. **design plan** — *which* compilation strategies to apply. A declarative specification declaring layers of primitives that select subsets of the Blueprint (annotated topology) and derive new state: meshing patterns, IP allocation pools and strategies, protocol overlay construction, policy rules, and assertion constraints. The design plan does not duplicate what is already in the topology; rather, it names the *design rules* that transform topology-level annotations into device-level state.

The relationship is analogous to a traditional compiler: the Blueprint (annotated topology) is *source code* and the design plan is a *compilation profile* (flags such as `-O2` or target architecture). The source code carries the program's semantics; the compilation profile selects *how* those semantics are lowered. Similarly, the topology carries network intent; the design plan selects which primitives lower that intent into device-level state.

3. **Target backend** — *how* to turn the enriched model into per-device configuration. In practice this is a small directory containing (i) target-specific transforms that lower vendor-neutral intent into platform-specific structure, and (ii) templates that serialise that structure into concrete CLI or configuration syntax. Most organisations author this once per platform and reuse it across sites.

> **Note**
>
> **Where does intent live?** The boundary between topology annotations and design plan declarations is a design choice that operators can tune. At one extreme, the topology carries only physical inventory (hostnames and cabling) and the design plan encodes all design rules. At the other, the topology carries rich intent (address pools, protocol parameters, zone memberships) and the design plan is a thin strategy selector. NetCfg supports both styles; the general trajectory is toward richer topologies with thinner design plans, because this keeps intent closer to the source-of-truth and reduces the coupling between the design plan and a specific topology's node schema.

4.1.1 Canonical annotation vocabulary

Table 2 lists the well-known annotation keys that NetCfg recognises on topology nodes. Operators may attach *any* key-value pair; the keys below receive special treatment from selectors, primitives, or the rendering engine.

Table 2. Well-known topology annotation keys. Keys marked with ⋆ are computed by the selector engine and need not appear in the topology file. All other keys are authored by the operator (or imported via `netcfg sync pull`).

Key	Type	Consumed by
`hostname`	string	All primitives (node identity)
`role`	string	Selectors (`MATCH (n) WHERE n.role ='n.sp ine'`)
`site`	string	Selectors, multi-site design plans
`device_os`	string	Template selection, vendor transforms, QoS constraints
`management_ip`	string	Inventory; passed through to templates
`zone`	string	`build_zone_policy`, zone-based assertions
`platform`	string	Hardware inventory transforms
`vrf`	string	`provision_ips` (routing context separation)
`disabled`	bool	Selectors (exclude nodes from processing)
Computed selector variables ⋆		
`id`	integer	Node identifier in the graph
`layer`	string	Current graph layer (e.g. `physical`, `bgp`)
`degree`	integer	Number of adjacent edges
`is_leaf`	bool	True when `degree == 1`
`neighbors`	list	Adjacent node identifiers

Any key not listed above is stored in the node's metadata map and remains accessible to selectors and templates. For example, an operator might annotate nodes with `trust_level: high` or `loopback_pool: 10.255.0.0/24`; these values are available to NetCfg queries and MiniJinja templates without any schema changes.

4.1.2 Thin design plan vs. rich topology

The boundary between topology annotations and the architectural plan is a design choice. At one extreme, the topology carries only physical inventory (hostnames and cabling) and the plan encodes all design rules. At the other, the topology carries rich intent (address pools, protocol parameters, zone memberships) and the plan is a thin strategy selector.

NetCfg supports both styles, but the general trajectory is toward *richer topologies with thinner plans*. This keeps intent closer to the source-of-truth and reduces the coupling between the design rules and a specific topology's node schema.

In a "fat plan" style, the design rules must name individual nodes and inject properties directly—tightly coupling the plan to a specific topology size. In a "thin plan" style, the topology carries its own identity (roles, sites, platforms), and the design rules operate purely on *semantic scopes* (e.g., "all edge routers"). The same architectural plan works unchanged if a fourth router is added to the topology.

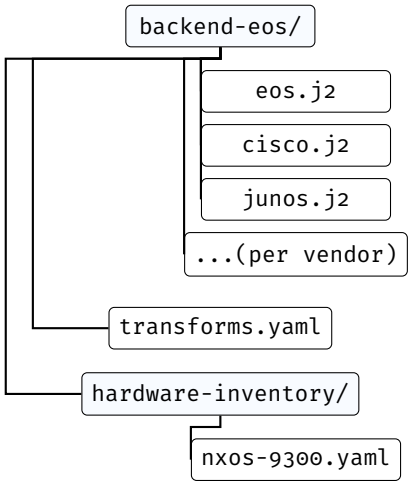


Figure 8. Target backend package structure. Templates render per-vendor syntax; `transforms.yaml` declares TSDM lowering rules; `hardware-inventory/` maps chassis models to slot layouts. Most organisations author this once per platform and reuse it across sites.

The Blueprint (annotated topology) is the primary authoring surface: it captures the network's identity, structure, and design intent. The design plan selects compilation strategies; the mapping document and templates are typically authored once per vendor platform and reused across sites.

With the data model established, this section turns to the design plan itself: how operators express intent, policy constraints, and design invariants. §?? then catalogues the concrete syntax.

4.2 The five concerns of a network design plan

Every design plan addresses five orthogonal concerns that must be mapped from the architect's intent into the final configuration of the network. Table 3 shows how each maps to a DSL construct, the design question it answers, and its ultimate configuration impact.

Table 3. The five concerns of a network design plan and their mapping to configuration.

Concern	DSL construct	Question answered	Configuration impact
Population naming	`groups:`	Which devices belong to which roles?	Determines target nodes for rendered configuration stanzas.
Topology intent	`layers:` / primitives	How should they be connected and what state should they carry?	Generates interfaces, IPs, and routing protocol sessions.
Policy constraints	routing/access/prefix-list primitives	What traffic rules should be enforced?	Synthesizes route-maps, prefix-lists, and ACLs.
Design invariants	`assertions:` + `rules:`	What properties must always hold?	Provides pre-flight validation (no direct config output).
Change safety	`blast_radius:`	How much change is acceptable per commit?	Acts as deployment pipeline gates (no direct config output).

The key insight is that every concern operates on the *network model* —the layered graph described in §3. The operator declares populations, intent, policy, and invariants; the synthesis engine maps these into graph mutations that progressively enrich the model's nodes, ultimately generating the final configuration artifact for each device.

4.3 From plan to network model: declarative composition

§4.4 introduced the five primitive tiers and their mutation semantics. This subsection shows how those tiers interact when composing a design plan.

`provision_ips` iterates edges to determine which node pairs need addresses, but writes `src_ip` and `dst_ip` onto the respective *nodes*—not onto the edge itself. `build_protocol_layer` clones selected nodes into a new graph layer and merges protocol configuration into them via deep merge. Each primitive sees the enriched model left by its predecessors, but never needs to know what they did.

: Node-Centric State

All per-device state is written to nodes via `DeviceContext`. Edges carry no mutable state; they exist solely to express relationships between nodes. This invariant simplifies the mapping stage: to produce a device's configuration, the mapping evaluator reads exactly one node.

Topology projections are the mechanism that links intent to the graph. Rather than binding configuration templates to device names via ad-hoc scripts, NetCfg uses declarative queries (NTE-QL) to project physical node populations into logical layers. For example, a design rule selects all `role == 'edge'` nodes and projects them into a BGP layer. This layer-scoping ensures that protocol design rules only operate on relevant network segments, treating the network as a coherent mathematical structure rather than a collection of independent text files.

4.4 Design plans compose the model declaratively

A design plan organises primitives into ordered *layers*. Each layer declares a set of primitives; each primitive selects a subset of the graph and enriches the nodes it touches. The result is a progressively richer model, as Figure 9 illustrates.

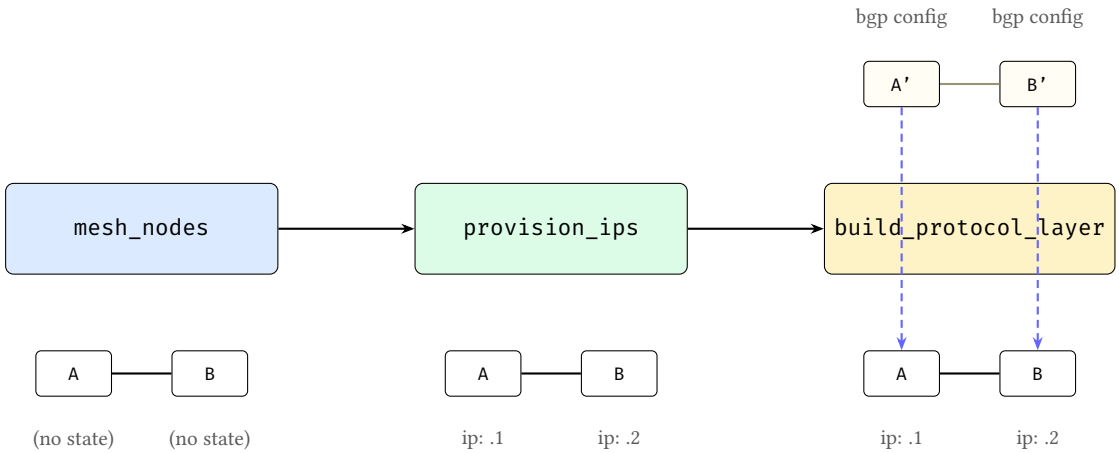


Figure 9. Progressive model enrichment. Left: `mesh_nodes` establishes edges between bare nodes. Centre: `provision_ips` annotates nodes with IP addresses. Right: `build_protocol_layer` clones nodes into a protocol layer and deep-merges a protocol overlay into the clones; dashed arrows show the parent mapping back to the physical devices. At each stage the model grows richer; no primitive needs to know what others have done.

Three properties make this composition model work:

1. **State lives on nodes, not edges.** Edges define relationships—physical links, peering sessions, policy bindings. All mutable, renderable state is written to the node at each end of an edge. Edges may carry static relationship metadata (e.g., labels), but do not carry configuration-bearing state.
2. **Primitives compose, not override.** Each primitive enriches the model additively. No primitive needs to know what other primitives have done; the model accumulates state layer by layer.
3. **Primitives are typed model transforms.** Each primitive belongs to one of five tiers, distinguished by what it mutates: *Topology* (new nodes and edges), *Allocation* (IP addresses, ASNs, VLANs), *Policy* (firewall rules, route-maps), *Overlay* (VXLAN VNIs, EVPN route-targets), or *Meta* (pipeline annotations and gates). Primitives never name specific devices; they reference *groups* and *selectors* (graph queries that resolve against the current model state), so the same design plan can operate on topologies of different sizes.

4.5 Mini worked example (conceptual)

The following tiny design plan fragment illustrates the layered-model flow without requiring any knowledge of underlying programming languages, template engines, or internal evaluation internals.

Listing 1. A minimal design plan fragment (layered model + design rules).

```yaml
layers:
  - name: physical
    primitives:
      - type: mesh_nodes
        selector: "MATCH (n) WHERE n.role =='core'"
        mesh_type: full

      - type: provision_ips
        selector: "MATCH ()-[e]->()"
        pool: 10.0.0.0/24
        subnet_size: 30

  - name: bgp
    requires: [physical]
```

```yaml
    primitives:
      - type: build_protocol_layer
        selector: "MATCH (n) WHERE n.layer =='physical' && n.role =='core'"
        layer: bgp
        clone_underlying: true
        config: { bgp: { enabled: true } }

      # With clone_underlying: true, BGP overlay edges are derived
      # automatically from the physical topology.  No separate
      # session primitive is needed.

assertions:
  - name: core-has-link-address
    severity: error
    select: "MATCH (n) WHERE n.role =='core'"
    check: { type: field_exists, field: src_ip }
```

Conceptually, this executes as follows:

1. **Start with an Blueprint (annotated topology)** containing core routers as physical nodes.
2. **`mesh_nodes`** creates physical edges between those nodes.
3. **`provision_ips`** examines the edges to decide which links need subnets, then writes the resulting link addresses onto the endpoint *nodes* (not onto the edges).
4. **`build_protocol_layer`** clones the physical core nodes into a BGP overlay layer and records a parent mapping (`_source_id`) back to the physical devices. The protocol overlay config is deep-merged into the clones.
5. **Overlay edges encode protocol intent.** With `clone_underlying: true`, the BGP overlay begins with derived connectivity copied from the physical layer. The synthesis engine then reads that overlay graph and writes per-device neighbour stanzas onto the participating nodes.
6. **Assertions** verify the model is well-formed (here: every core device has a link address) before any vendor-specific configuration is produced.

The key point is that the operator never names a specific interface or assigns an IP address directly. The design plan describes intent, and the synthesis engine derives concrete per-device state from the layered topology relationships.

4.6 The select–enrich–validate pattern

Every primitive follows the same three-phase pattern (Figure 10):

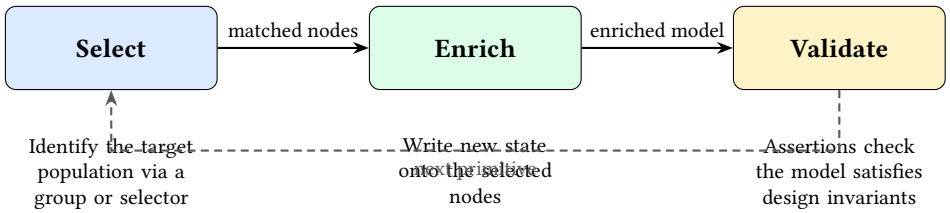


Figure 10. The select–enrich–validate cycle. Every primitive selects a population of nodes or edges and enriches them with new state. Validation is expressed as assertions over the model and may run as an explicit checkpoint, a layer barrier, or a final gate before rendering.

- **Select.** A group reference or NetCfg query identifies which nodes (or edges) the primitive will operate on. The selector is evaluated against the current graph state, so earlier primitives' enrichments are visible to later selectors.
- **Enrich.** The primitive performs a typed model mutation on the selected population. Depending on the primitive's tier, this may produce new graph structure (overlay nodes and edges), allocated identifiers (IP addresses, ASNs, VLAN IDs), compiled policy objects (firewall rules, route-maps, prefix-lists), or protocol stanzas (VXLAN VNIs, EVPN route-targets, BGP peering parameters). State always accumulates; primitives never delete or overwrite each other's contributions.

- **Validate.** Assertions verify that the model satisfies the operator's invariants (e.g., "every node has a link address" or "no two links share a subnet"). Assertions may run immediately (as checkpoints), at the end of a layer, and again as a final gate before any configuration is generated.

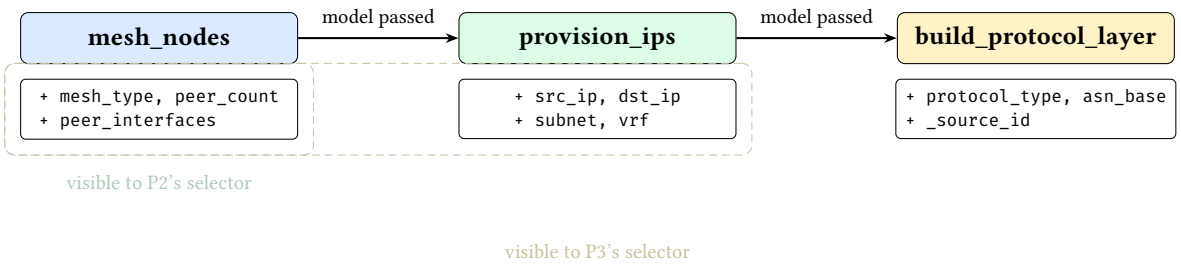


Figure 11. Selector visibility timeline. Each primitive's selector evaluates against the *current* graph state, which includes all fields written by earlier primitives. State accumulates monotonically.

Validation checkpoints. NetCfg supports three complementary validation styles, all expressed as assertions over the current model state: (1) *mid-pipeline checkpoints* (via `assert_state`) that fail fast after a critical enrichment step; (2) *layer barriers* that run after a layer's primitives execute; and (3) *final gates* that run before any configuration is rendered. Conceptually, all three ask the same question—"does the model satisfy the design invariants?"—but they fire at different times for faster feedback.

This uniform pattern means that learning one primitive teaches the operator the shape of every primitive: declare a selector, declare the desired state, and optionally assert invariants. The formal definitions and concrete DSL syntax follow in §3 and §**??**.

4.6.1 Structuring the compilation plan

The plan organizes design rules into ordered compilation layers. Each layer executes sequentially, allowing logical topologies to be built on top of physical structures.

Listing 2. Design Plan top-level schema.

```
version: 1                        # schema version (required)
imports: [shared-rules.yaml]      # merge other design plans
layers:                           # ordered compilation layers
  - name: physical
    requires: []                  # layer dependencies
    primitives: [...]             # typed model transforms
assertions: [...]                 # post-execution design checks
rules:                            # composed Cypher queries
  is_core: "role == 'core'"
groups:                           # named node sets
  core_routers: { selector: "MATCH (n) WHERE n.role =='core'" }
blast_radius: [...]               # change safety policies
transforms: [...]                 # vendor-specific adaptation
hardware_library: { ... }        # chassis/linecard definitions
```

Three validation passes run at parse time:

1. **Schema validation**: field types, non-empty names, integer ranges. Unknown schema keys are rejected at parse time, catching typos before execution.
2. **NetCfg query compilation**: each selector string is compiled into an evaluable query program, surfacing syntax errors before execution.
3. **Layer ordering validation**: a forward scan ensures every `requires` entry names a previously-declared layer. This catches ordering bugs at parse time.

The optional `imports` field lists relative file paths to other design plan files. At parse time, layers and assertions from imported files are merged into the importing design plan, enabling reusable assertion libraries (e.g. a standard linting ruleset shared across projects).

Warning

Duplicate layer names are permitted. Multiple `LayerSpec` blocks with `name: input` are common in practice—each block adds primitives to the same layer. The `requires` mechanism enforces inter-layer ordering, not intra-layer uniqueness.

4.6.2 Selector DSL (NTE-QL Surface Syntax)

The NetCfg Query Language (NTE-QL)

NTE-QL is the graph-traversal language that drives NetCfg. Traditional automation uses explicit `for` loops and `if` statements in Python or Jinja; NTE-QL replaces these with mathematical predicates over nodes and edges (e.g., "all spine routers in London"). The synthesis engine translates each NTE-QL selector into an optimised graph traversal against the NTE engine. NTE-QL parses directly into the graph traversal AST. (NTE-TR-001 §Query System).

Selectors identify which nodes or edges a primitive operates on. Conceptually, a selector is simply a logical predicate over the current model: "for each node (or edge), should this primitive apply here?" Rather than treating configuration as flat text, the query language treats the network as a property graph. The concrete surface syntax is heavily inspired by Cypher, allowing engineers to express topological relationships mathematically. Under the hood, these selector queries are dispatched to the NTE (Network Topology Engine), which natively evaluates the Cypher-like graph traversals in Python before returning the matched node sets back to the synthesis engine:

Listing 3. Selector syntax examples.

```yaml
# Select all nodes
selector: "all()"

# Select nodes by property (e.g. region, role, type)
selector: "MATCH (n) WHERE n.region =='eu-west'"
selector: "MATCH (n) WHERE n.role =='core' && n.layer =='input'"
selector: "MATCH (n) WHERE n.type =='Router' and n.region =='us-east'"

# Select edges
selector: "MATCH ()-[e]->() WHERE e.src >=0"
selector: "MATCH ()-[e]->()"
```

The selector parser:

1. Identifies whether the selector targets `nodes` or `edges` and extracts the bracketed predicate.
2. Normalises minor syntax differences (e.g. `and` vs `&&`) so the predicate is unambiguous.
3. Compiles the predicate into an evaluable program.

At evaluation time, the selector ranges over all nodes (or edges) in the topology and returns the subset for which the predicate is true. This means selectors are always evaluated against the *current* model state: earlier primitives can add fields that later selectors use.

Mechanically, the synthesis engine binds each entity's properties as variables (`id`, `type`, `layer`, plus all `data_json` fields, flattened and accessible via dot notation) and evaluates the compiled predicate.

Note

Why missing fields are not errors. Because the model is layered, not every node has every field: a physical node will have interface and addressing facts, while a protocol node will have protocol-specific facts. Selectors must therefore tolerate heterogeneous populations. Mechanically, undeclared variable references (e.g., querying a field that some nodes lack) silently evaluate to `false` rather than producing an error. This lets a single selector range over mixed layers while still precisely matching only the nodes that carry the relevant facts.

For IP-valued fields, the selector also binds a `{field}_len` variable containing the prefix length, enabling selectors like `MATCH (n) WHERE n.subnet_len == 30`.

> **Note**
>
> **Layer-aware selector patterns.** Because the model is layered, selectors typically constrain both *what* a node is (role/type) and *where* it lives (layer). Common patterns include:
>
> ```
> # Physical devices only
> selector: "MATCH (n) WHERE n.layer =='physical' && n.role =='core'"
>
> # Protocol overlay nodes only (e.g. BGP layer)
> selector: "MATCH (n) WHERE n.layer =='bgp' && n.role =='core'"
>
> # Edges within a layer (physical links, protocol sessions)
> selector: "MATCH ()-[e]->() WHERE e.layer =='physical'"
>
> # Guard against heterogeneous fields (match only nodes that have the facts)
> selector: "MATCH (n) WHERE n.layer =='bgp' && has(node.bgp) && node.bgp.n.enabled ==true"
> ```

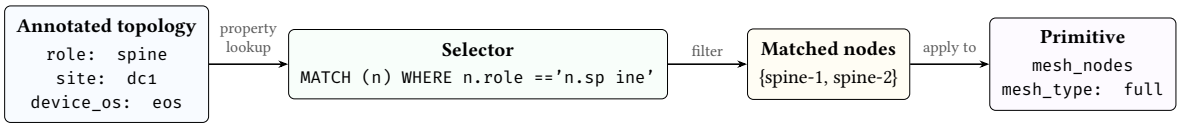


Figure 12. Selector-to-annotation data flow. Topology annotations are the values that selectors query; the selector filters nodes into a matched set; the primitive then operates on that set. The topology annotations are the “input language” and the selector is the bridge between topology intent and design plan strategy.

4.6.3 Named groups

The `groups:` section defines reusable named node or edge sets. Group names are prefixed with `$` when referenced in selectors. Two forms are supported:

Listing 4. Group definitions: shorthand and full form.

```
groups:
  # Shorthand: just a selector string (e.g. by role)
  core_routers: "MATCH (n) WHERE n.role =='core'"
  access_switches: "MATCH (n) WHERE n.role =='access'"

  # Full form with kind and kind-specific fields
  dmz:
    selector: "MATCH (n) WHERE n.zone =='dmz'"
    kind: security_zone
    trust_level: 10
```

Groups are resolved before primitives execute. The selector parser expands `$group_name` references inline, and cycle detection rejects circular group dependencies at parse time. When design plans import other files, groups from imported files are merged into the importing design plan’s namespace (name collisions are last-writer-wins).

The union operator (`|`) combines groups:

```
  all_devices: $core_routers | $access_switches
```

This creates a set containing all nodes matching either group.

When a group has `kind: security_zone`, an optional `interface_selector` field may be supplied to express interface-level zone membership. This selector targets edges rather than nodes, enabling fine-grained policies where different interfaces on the same device belong to different zones:

```
  dmz:
    selector: "MATCH (n) WHERE n.zone =='dmz'"
    kind: security_zone
    trust_level: 10
    interface_selector: "MATCH ()-[e]->() WHERE e.intf_role =='dmz'"
```

4.6.4 Named objects

The top-level `objects:` section defines reusable address and service objects. These are referenced in zone policy and NAT policy rules using the `$` prefix (e.g. `$rfc1918`, `$web_services`). Both sub-maps default to empty, so omitting `objects:` is backward-compatible.

Listing 5. Named address and service objects.

```
objects:
  addresses:
    rfc1918:
      prefixes:
        - "10.0.0.0/8"
        - "172.16.0.0/12"
        - "192.168.0.0/16"
    corp_v6:
      prefixes_v6:
        - "2001:db8::/32"
  services:
    web_services:
      entries:
        - { protocol: tcp, port: 80 }
        - { protocol: tcp, port: 443 }
    dns:
      entries:
        - { protocol: udp, port: 53 }
        - { protocol: tcp, port: 53 }
```

`AddressObject` supports dual-stack addressing via separate `prefixes` (IPv4) and `prefixes_v6` (IPv6) fields. Prefixes are stored as strings and validated to `ipnet::IpNet` at object registry build time.

`ServiceObject` contains one or more `ServiceEntry` pairs of `protocol` and `port`. In zone policy or NAT rules, use `service: $web_services` in the `match:` block to reference a named service object.

5 Error Model

NetCfg uses typed error enums with `thiserror` derivation and `miette` diagnostic annotations. Table 4 summarises the error types.

Table 4. Error enums and their variants.

Enum	Variants	Context
`ParseError`	`YamlSyntax`, `Validation`, `Deserialize`, `LayerOrdering`	design plan parsing
`SelectorError`	`InvalidSyntax`, `Compile`, `Evaluate`, `NonBooleanResult`, `Polars`, `Json`, `TopologyAccess`	Selector compilation and evaluation
`PrimitiveError`	`Validation`, `Execute`, `Selector`, `Topology`, `Graph`, `Polars`, `Json`	Primitive execution
`ExecuteError`	`Parse`, `Primitive`, `Shadow`, `Plan`, `Assertion`	Top-level executor
`ShadowError`	`Clone`, `Commit`, `Validation`	Shadow topology lifecycle
`AssertionError`	`Failed`, `Selector`	Assertion evaluation
`DiffError`	`TopologyAccess`, `ParseError`	Configuration diffing
`RenderError`	`Io`, `Jinja`, `Json`, `MissingTemplate`	Template rendering

`ExecuteError` variants carry `miette::Diagnostic` annotations with diagnostic codes (e.g., `netcfg::parse`, `netcfg::primitive::execution_failed`, `netcfg::assertion`) enabling structured error handling in CI pipelines.

`ParseError::YamlSyntax` includes a source span (`miette::SourceSpan`) pointing at the offending line, enabling IDE-quality error rendering.

6 Editor and Service Integration

6.1 Language Server (LSP)

The `netcfg-lsp` crate provides a Language Server Protocol implementation built on `tower-lsp` and Tokio. It communicates via stdin/stdout and offers three capabilities:

1. **Real-time diagnostics.** On file open, change, and save, the server parses the design plan and maps errors to LSP diagnostics with line-column precision. Text parse errors (which embed location as free text) are converted to line/column positions via pattern matching. Diagnostics are published with source label `ank_netcfg` and severity `ERROR`.
2. **Primitive auto-completion.** When the cursor follows a `type:` token (detected by a suffix heuristic), the server offers completion items for all twenty-three primitives, each with a one-line docstring. Trigger characters are space and colon.
3. **Document synchronisation.** Open documents are tracked in a `DashMap<String, String>` (lock-free concurrent map). Full-text sync (`TextDocumentSyncKind::FULL`) is used for correctness. On close, the document is removed and diagnostics are cleared.

6.2 REST API Microservice

The `netcfg-api` crate exposes the synthesis engine as a stateless HTTP service via `axum`, enabling CI/CD pipelines and enterprise orchestrators to compile design plans without a local `netcfg` binary.

Endpoint: `POST /compile` (port 3000).

Table 5. Compile endpoint request and response schema.

Direction	Field	Type	Description
Request	`topology_zip_base64`	String	Base64-encoded `.nte` archive
Request	`design plan_yaml`	String	Raw Design Plan
Response	`status`	String	`success` or `error`
Response	`configs`	Map<String, String>	Hostname → rendered config
Response	`device_ir`	Map<String, JSON>	Hostname → DeviceContext IR
Response	`warnings`	Vec<String>	Compilation warnings

The service executes in a strict sandbox mode: `inject_secrets` and `wasm_plugin` primitives are disabled by default to prevent remote exfiltration of environment variables. The compilation pipeline decodes the base64 payload into a temporary file, loads the topology, parses the design plan, executes via `Executor::apply()`, and extracts per-device IR. All processing is in-memory with no persistent state; the service scales horizontally behind a load balancer.

7 Limitations

7.1 `edge_properties` not applied (`mesh_nodes`)

Description. The `MeshNodesSpec.edge_properties` field is accepted by the design plan parser but intentionally not applied. Applying it requires an edge DataFrame API on `nte_topology::Topology` equivalent to `set_dataframe/update_dataframe` but keyed on $(source\_id, target\_id)$ pairs.

Workaround. Edge properties can be encoded as node metadata on both endpoints (e.g., `link_bandwidth` as a per-node field keyed by peer ID).

Planned resolution. Awaiting `nte-topology` ≥0.2 edge DataFrame support. A tracking test (`edge_properties_deferred_until_nte_edge_dataframe`) is present in the test suite.

7.2 Mapping Structural Model inspection not implemented

Description. The `inspect -ast` command can parse and display design pla structured logical plans but does not yet support mapping documents. Attempting to inspect a mapping file prints "Mapping Structural Model visualization coming soon."

Workaround. Inspect the mapping YAML directly. The format is simple (a `rules` list with `selector` and `stanza` fields).

Planned resolution. When the mapping DSL grows beyond simple selector/stanza pairs, Structural visualisation will be added.

7.3 Shadow validation is a placeholder

Description. The `ShadowTopology::validate()` method currently only checks that the shadow did not become empty. It does not perform structural integrity checks (dangling edges, orphaned protocol nodes, etc.).

Workaround. Design-rule assertions can catch many structural problems. Use `min_edges` and `field_exists` assertions as guardrails.

Planned resolution. Integrate with the NTE policy validation framework when available.

7.4 No intra-primitive rollback

Description. If a primitive fails partway through execution (e.g., `provision_ips` exhausts its pool after allocating half the edges), the shadow topology retains the partial mutations. The pipeline aborts (returning an error), and the shadow is discarded, but there is no mechanism to roll back individual primitives within a shadow.

Workaround. This is usually not a problem: the entire shadow is discarded on error, so no partial state reaches production. The limitation matters only for debugging: the error message may not reflect the full extent of partial mutations.

Planned resolution. No current plan. Intra-primitive rollback would require snapshotting the shadow before each primitive, which doubles memory usage.

7.5 `generate` CLI bypasses mapping document

Description. The `generate` CLI command renders configs directly from `DeviceContext` via templates, bypassing the mapping document. The full pipeline (with mapping evaluation and DeviceIR) is implemented in `config_gen::generate_configs()` but the CLI does not expose it yet.

Workaround. Use the `generate_configs()` function directly from Rust code for the full pipeline with mapping support.

Planned resolution. Add `–mapping` flag to the `generate` CLI command.

7.6 No incremental compilation

Description. Every invocation recompiles the entire topology from scratch. The diff engine can identify which devices changed, but the pipeline cannot skip unchanged devices during compilation.

Workaround. Use `ci-run -json` to compute the diff, then selectively push only the changed configuration files.

Planned resolution. Under investigation. Incremental compilation requires caching intermediate state (the desired topology) between runs and detecting which design plan changes invalidate which nodes.

7.7 No runtime verification or drift detection

Description. NetCfg is a generation-time tool. It produces configuration artifacts but does not push them to devices, monitor compliance, or detect post-deployment configuration drift. Closed-loop verification requires integration with external tools.

Workaround. Periodically regenerate configurations from the canonical design plan and diff against deployed state using `ci-run -json`. Use Batfish [10] or gNMI-based telemetry for runtime compliance checking.

7.8 Shadow topology cloning cost

Description. The transactional shadow is created via a deep clone operation, which is $O(n)$ in topology size. For 10,000-node topologies this adds ~200 ms of overhead.

Workaround. Not typically a bottleneck, but for interactive workflows the cost is noticeable. Future work may explore copy-on-write graph snapshots in the NTE engine.

7.9 WASM plugin is experimental

Description. The `wasm_plugin` primitive validates module compilation and symbol resolution but does not implement full linear memory I/O for JSON data exchange between the Rust host and WASM guest. The current implementation is proof-of-concept only.

Workaround. Use `custom_primitive` with nested built-in primitives for extensibility. For complex transformations, implement them as Rust primitives.

Planned resolution. Full C-ABI memory allocation between host and guest is required for production use. See §??.

7.10 Mapping document limitations

Description. Two limitations affect the mapping workflow: (1) the `inspect -ast` command does not yet support mapping documents (§7.2); and (2) there is no auto-generation of mapping rules from OpenConfig YANG schemas or device inventory data—operators must author the mapping document by hand, which requires knowledge of `DeviceContext` field names and stanza structure.

Workaround. The mapping format is intentionally simple (a `rules` list with `selector` and `stanza` fields). Example mappings ship with the worked examples (`three-site-mesh-mapping.yaml`, `evpn-vxlan-mapping.yaml`).

7.11 NSoT push not implemented (`netcfg sync push`)

Description. The `netcfg sync push` subcommand is defined in the CLI but returns an error when invoked. Pushing intent back to an NSoT system requires a write-capable provider interface (e.g. NetBox REST API mutations) that has not yet been implemented.

Workaround. Export the compiled topology to an NTE archive (`sync pull -o`) and ingest it into the NSoT manually.

Planned resolution. A `NetBoxWriter` provider will be added alongside the existing `NetBoxProvider` read path.

References

[1] S. Shenker, "The future of networking, and the past of protocols," in *Open Networking Summit*, vol. 20, Palo Alto, CA: Open Networking Foundation, 2011, pp. 1–30.

[2] P. Gill, N. Jain, and N. Nagappan, "Understanding network failures in data centers: Measurement, analysis, and implications," in *ACM SIGCOMM computer communication review*, vol. 41, New York, NY, USA: ACM, 2011, pp. 350–361.

[3] J. Zhu et al., "MALT: Multi-abstraction layer topology," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, Santa Clara, CA: USENIX Association, 2020, pp. 313–328.

[4] OpenConfig Working Group, *OpenConfig*, https://openconfig.net, 2023.

[5] M. Bjorklund, "The YANG 1.1 data modeling language," IETF, RFC 7950, 2016.

[6] NetBox Community, *NetBox: The source of truth for network automation*, https://netbox.dev/, 2023.

[7] Network to Code, *Nautobot: Network source of truth and automation platform*, https://nautobot.com/, 2023.

[8] Red Hat, *Ansible automation platform*, https://www.ansible.com/, 2023.

[9] Nornir Automation, *Nornir: Pluggable multi-threaded framework with inventory management to help operate collections of devices*, https://nornir.tech/, 2023.

[10] Intentionet, *Batfish: Network configuration analysis tool*, https://www.batfish.org/, 2023.

[11] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, IEEE, San Jose, CA: IEEE, 2004, pp. 75–86.

[12] S. Knight, *AutoNetKit NTE: The network topology engine*, 2026.

[13] J. Moy, "OSPF Version 2," IETF, RFC 2328, Apr. 1998.

[14] D. Oran, "OSI IS-IS intra-domain routing protocol," IETF, RFC 1142, Feb. 1990.

[15] Y. Rekhter, T. Li, and S. Hares, "A Border Gateway Protocol 4 (BGP-4)," IETF, RFC 4271, Jan. 2006.

[16] E. Rosen, A. Viswanathan, and R. Callon, "Multiprotocol Label Switching Architecture," IETF, RFC 3031, Jan. 2001.

[17] M. Mahalingam et al., "Virtual eXtensible Local Area Network (VXLAN): A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks," IETF, RFC 7348, Aug. 2014.

[18] A. Sajassi et al., "BGP MPLS-Based Ethernet VPN," IETF, RFC 7432, Feb. 2015.